LAB ASSIGNMENT 15.2

Lab 15 – Backend API Development: Creating RESTful Services with AI

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1: Setup FastAPI Backend

Prompt:

Create a basic FastAPI app with a root GET endpoint that returns a welcome message.

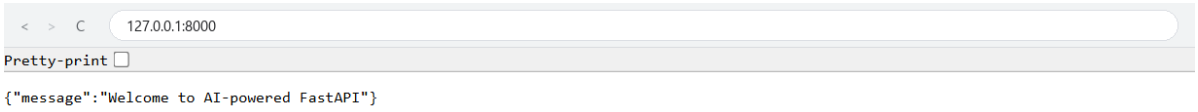
Code

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def read_root():
    """Return a welcome message."""
    return {"message": "Welcome to AI-powered FastAPI"}
```

Output



Justification

FastAPI creates the backend application. The GET endpoint at / returns a welcome message and confirms that the server is running.

Task 2: API – Create and Read Items

Prompt:

Add GET and POST endpoints to retrieve and create items using a Python list.

Code

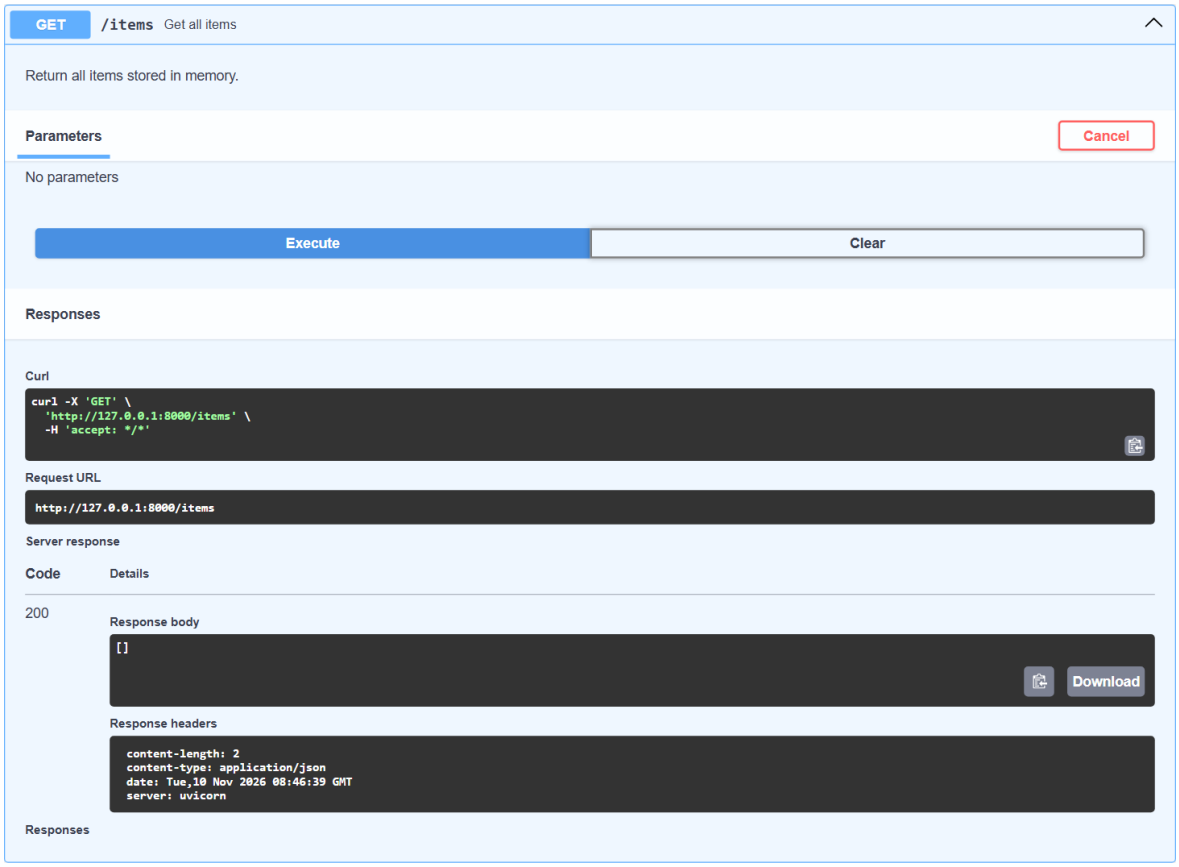
```
items = []


@app.get("/items")
def get_items():
    """Return all stored items."""
    return items


@app.post("/items")
def add_item(item: dict):
    """Add an item to the list."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output

GET /items :



POST /items :



Justification

The GET endpoint displays the items in the list. The POST endpoint accepts an item as JSON, stores it in the list, and returns a confirmation message.

Task 3: Update Existing Item

Prompt:

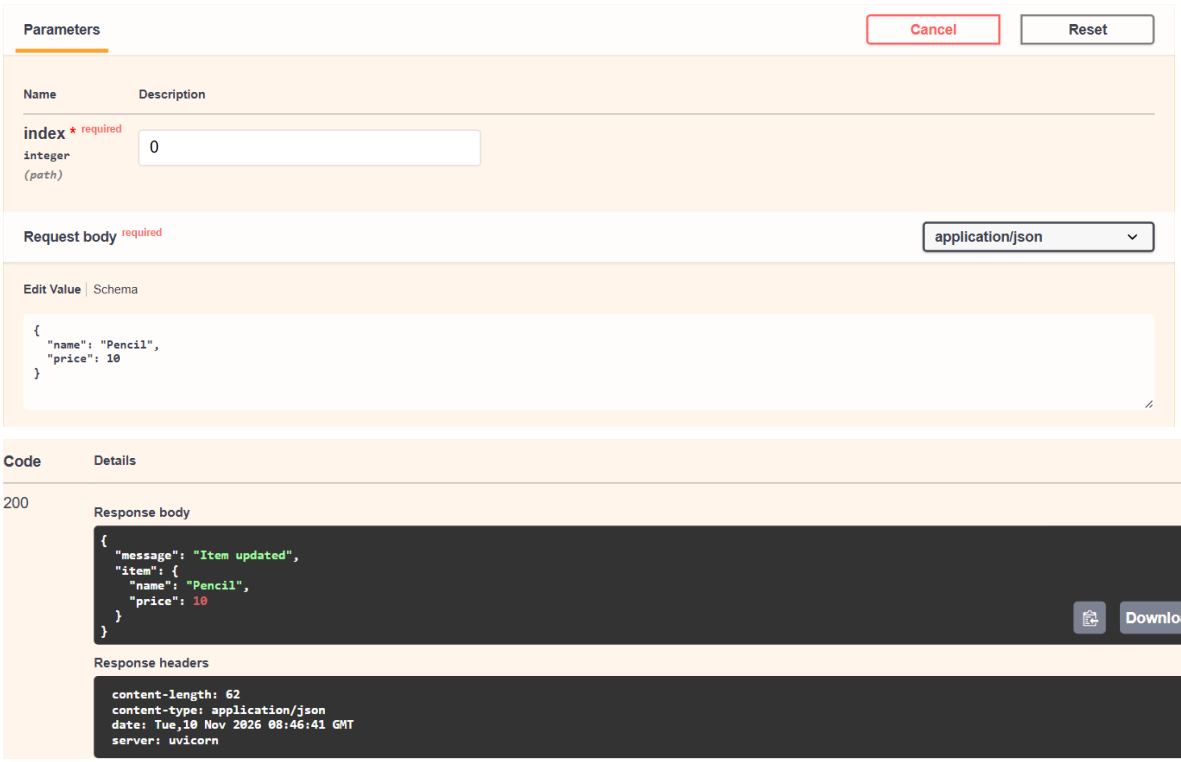
Create a PUT endpoint that updates an item by its zero-based index and reports when the item is not found.

Code

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """Update an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output

PUT /items:



Justification

The PUT endpoint replaces the item at the specified index. It returns a 404 error if the index does not refer to an existing item.

Task 4: Delete Item

Prompt:

Create a DELETE endpoint that removes an item by its index and returns the deleted item.

Code

```
@app.delete("/items/{index}")
def delete_item(index: int):
    """Delete an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output

DELETE /items/0 (200 OK):

Code | Details

200

Response body


```
{
  "message": "Item removed",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```


Download

Response headers


```
content-length: 62
content-type: application/json
date: Tue,10 Nov 2026 08:46:41 GMT
server: uvicorn
```


DELETE /items/0 after removal (404 Not Found):

Code | Details

404 *Undocumented*

Error: Not Found

Response body


```
{
  "detail": "Item not found"
}
```


Download

Response headers


```
content-length: 27
content-type: application/json
date: Tue,10 Nov 2026 08:46:42 GMT
server: uvicorn
```


Justification

The DELETE endpoint removes the selected item from the list and returns the removed item. Invalid indexes produce a 404 error.

Task 5: Add Auto-Generated Documentation

Prompt:

Add comments, docstrings, endpoint summaries, and Swagger documentation to the FastAPI endpoints.

Code

```
from fastapi import FastAPI, HTTPException

app = FastAPI(
    title="Items CRUD API",
    description="API for creating, reading, updating, and deleting items",
    version="1.0.0"
)

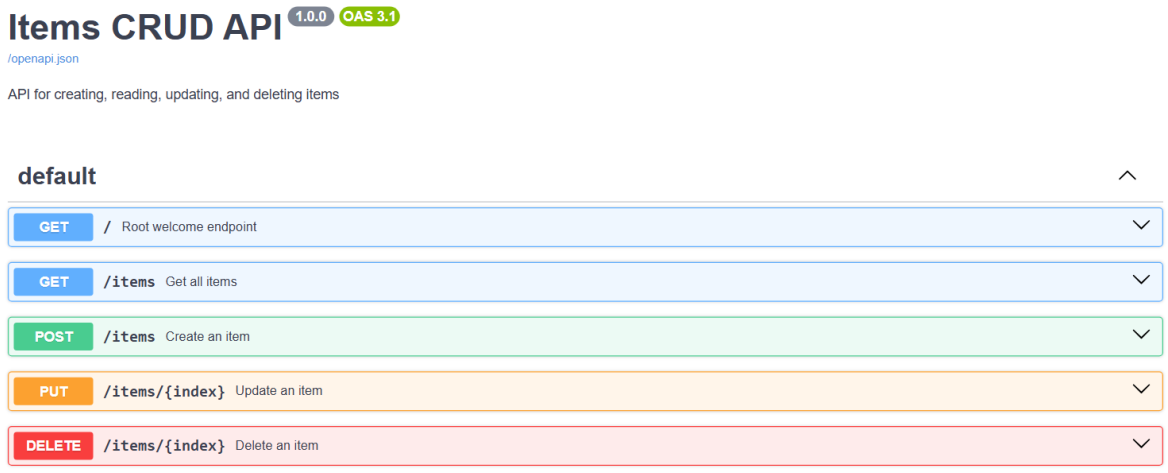

@app.get("/items", summary="Get all items")
def get_items():
    """Return all items stored in memory."""
    return items


@app.post("/items", summary="Create an item")
def add_item(item: dict):
    """Create and return a new item."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

Summaries and docstrings are also attached to the PUT and DELETE endpoints in Lab15/main.py.

Output

Swagger UI (http://127.0.0.1:8000/docs):



Justification

FastAPI reads endpoint definitions, type hints, summaries, and docstrings to generate interactive Swagger documentation automatically. The /docs page helps users test the API without needing a separate API client.

Complete Program Note

For a single runnable program, all endpoint definitions from Tasks 1–5 are combined into Lab15/main.py using from fastapi import FastAPI, HTTPException, with a shared app = FastAPI(...) instance and items = [] store. Run the server with: uvicorn main:app --reload. Note: The in-memory item list resets when the server restarts.